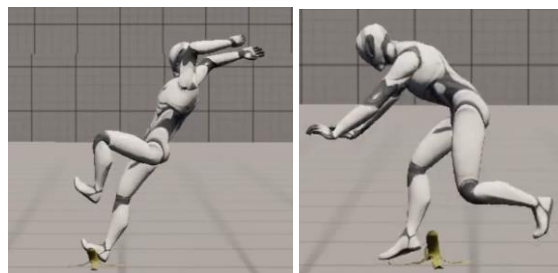


Assignment #03: Animation Project – Principles and Physics

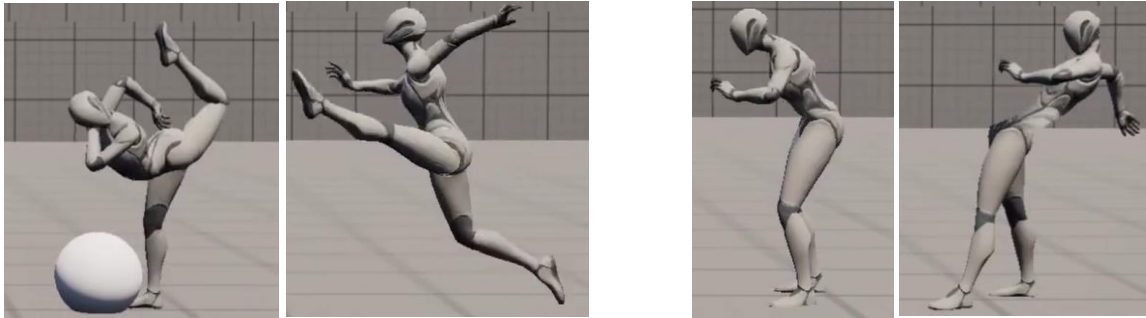
The animation depicts a character (SKM_Manny_Simple) entering the scene and stepping on a banana peel [1]. This leads them to temporarily lose their balance before regaining it and continuing forward. Seconds later, a ball enters the scene at high speed and strikes the character from behind, triggering a transition into ragdoll physics as the character collapses to the ground. The ball continues to bounce away, coming to a rest in the opposite direction of the ragdoll, after which a second character (SKM_Quinn_Simple) runs onto the scene and kicks the ball off-screen. After some time passes, the second character notices the ragdoll and, realising the consequence of their action, reacts with an exaggerated shocked pose.

Required feature: 4 Principles of Animation*Squash and Stretch*

During the slipping motion, the spine of the first character stretches slightly upward as balance is lost, before reaching downwards as they attempt to regain their balance.

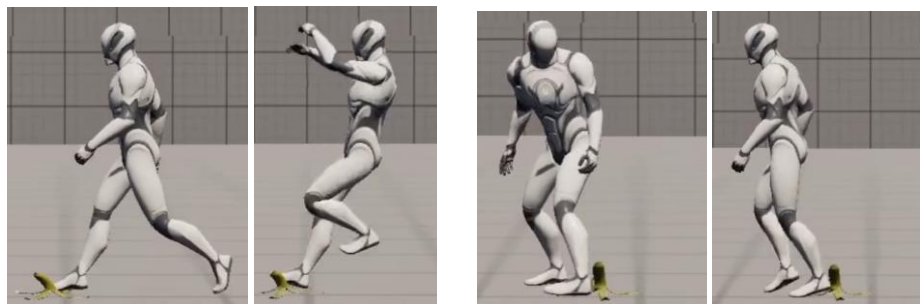


The ball demonstrates more overt squash and stretch behaviour. It stretches in the direction of motion as it travels toward the character, squashes upon impact with the back of their head, and continues to squash and stretch during subsequent bounces with decreasing intensity, reflecting energy loss.



Additionally, squash and stretch were applied to the second character during the wind-up of the kick and the exaggerated shocked pose, where body compression and extension add to their readability and expressiveness.

Anticipation



As the first character begins to slip, subtle shifts in posture signal the impending loss of balance. After regaining stability, the pause and glance toward the banana peel provide a moment of anticipation before continuing movement.



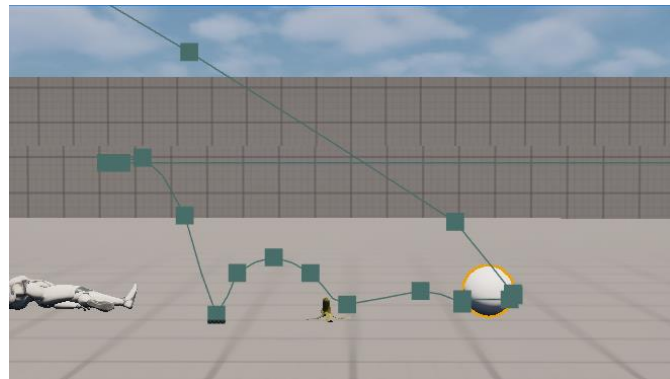
The second character demonstrates anticipation by winding up before kicking the ball. The leg is drawn back prior to the forward striking motion, the body shakes briefly to

convey the build-up in power, and the drawn back leg extends slightly higher moments before the kick is finally released.

Follow Through



When the first character transitions into ragdoll physics following the ball impact, the limbs continue moving due to inertia, with arms and legs lagging behind the torso before settling.

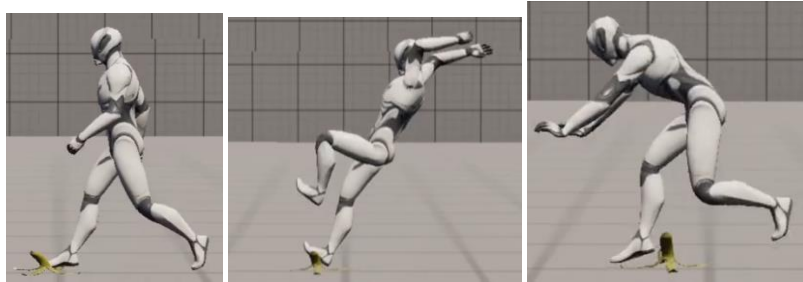


The ball also exhibits follow-through behaviour, bouncing and rolling away after impact with progressively reduced motion, indicating energy dissipation.



Additionally, the banana peel continues to move and rotate after being stepped on, acting as a secondary element that reinforces the slipping action. This continued motion supports the realism of the interaction and contributes to the overall sense of physical continuity.

Exaggeration



The slipping motion is amplified through large, unstable body movements, making the loss of balance more visually readable.



The final reaction of the second character is highly exaggerated, with a dramatic shocked pose that emphasises the realisation of what has occurred.



The ball's motion is also exaggerated in its speed and deformation, to increase the perceived impact and maintain a stylised tone consistent with the animation.

Physically based Animation Topic 1 – Ragdoll Physics

Background

Ragdoll physics is a technique used to simulate realistic character motion by replacing keyframed animation with physically driven behaviour. In this circumstance, the character is modelled as a system of rigid bodies connected by constraints that represent skeletal joints [2].

The motion of each rigid body is governed by Newton's Second Law:

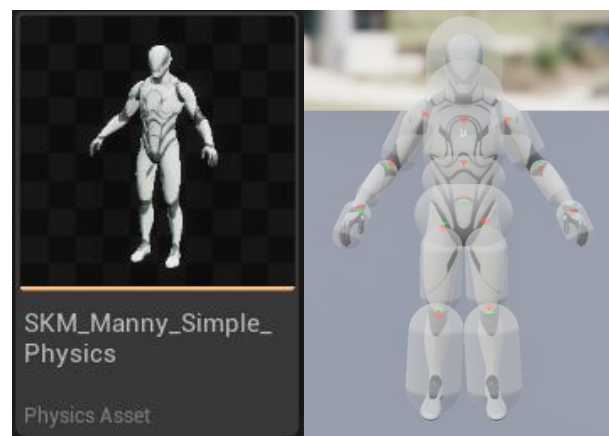
$$F = ma,$$

where the applied force determines acceleration based on mass. Gravity, collision impulses, and contact forces are all external factors that propel the system into motion.

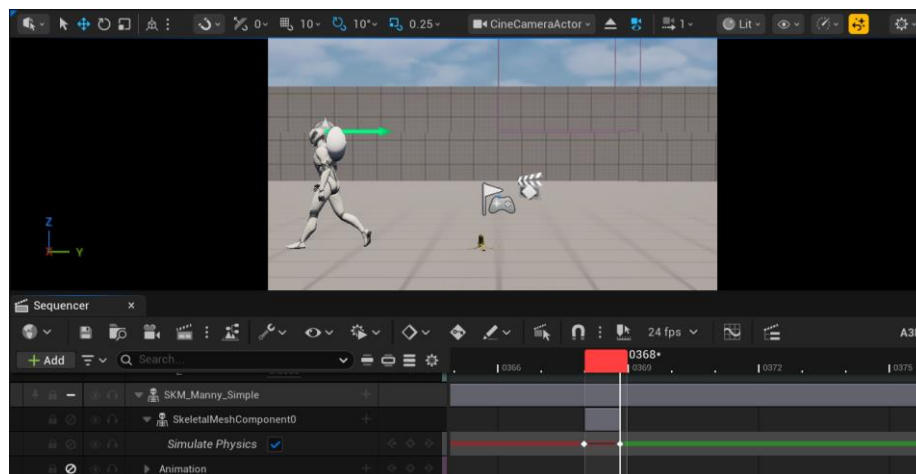
Constraints are used between rigid bodies to restrict relative movement and simulate anatomical constraints, such as hinge joints for elbows and ball-and-socket joints for shoulders [3].

To maintain stability in real-time simulation, damping is introduced to reduce oscillations, and iterative constraint solvers are used to enforce joint limits. Modern physics systems, such as Unreal Engine's Chaos system, approximate these interactions in real time using numerical methods [2].

Implementation and Investigation in Unreal



Ragdoll physics was implemented using the Physics Asset associated with the SKM_Manny_Simple skeletal mesh. This asset defines collision bodies for individual bones and constraints between them, enabling physically simulated motion.



In the sequencer, the transition from animation to physics was achieved by enabling “Simulate Physics” on the skeletal mesh component the moment the ball impacts the character. The animation track was trimmed at this frame to prevent conflicts between keyframed animation and simulation.

Parameter exploration was conducted to refine the behaviour. Increasing angular damping reduced excessive rotational motion of limbs, while adjusting constraint limits

prevented unrealistic joint bending. It was observed that higher damping improved stability but reduced responsiveness, showing the trade-off between physical realism and controllability.

Evaluation and Reflection

The ragdoll system generated natural secondary motion to a satisfactory degree, which in turn enhanced the realism of the collision. The transition from keyframed animation to simulation clearly demonstrated the influence of external forces on the character's motion. However, some instability occurred when constraints were insufficiently restricted, leading to unrealistic limb behaviour. This reflects a common limitation in real-time physics systems, where stability and accuracy must be balanced.

Ragdoll physics strongly supported the principle of follow-through, as the limbs continued moving after impact due to inertia. It also contributed to exaggeration by amplifying the visual impact of the collision. With more time, further refinement of constraint parameters and exploration of blending techniques between animation and physics would improve the smoothness and realism of the transition.

Physically based Animation Topic 2 – Niagara Particle Systems

Background

Particle systems are a technique used to simulate complex visual phenomena such as smoke, fire, dust, and impact effects using a large number of small elements called particles. Niagara operates using a system of emitters and modules. Emitters define how particles are spawned, including properties such as initial velocity, lifetime, and spawn rate, while modules update particle attributes over time, such as position, velocity, and scale [4]. These behaviours can be executed on either the CPU or GPU, allowing for efficient simulation of large numbers of particles in real time.

Particle systems are commonly used to enhance animation by visually representing forces and energy transfer, particularly in events such as impacts or explosions. While they may not always simulate full physical interactions, they provide an effective approximation that improves visual feedback and realism.

Implementation and Investigation in Unreal



The Niagara particle system preset “NS_Damage” was used to enhance the visual impact of the ball striking the back of the characters head. A burst emitter was used to generate particles upon collision, simulating an outward burst of sparks from the impact point.

The particle system was modified with its initial velocity directed away from the collision point to reflect the transfer of momentum. The size and spawn rate of the particles were also adjusted to maintain visibility without overwhelming the scene.

Several parameters were explored during development. Increasing the spawn rate produced a denser and more intense effect but reduced clarity; while decreasing it made the impact less noticeable. Adjusting initial velocity affected how far particles travelled from the origin, influencing the perceived strength of the collision. Shorter lifetimes resulted in a more dynamic burst, whereas longer lifetimes caused the particles to linger and appear less physically motivated.



Through experimentation, a balance was achieved between visual clarity and perceived physical energy, ensuring the particle effect complemented the animation without distracting from it.

Evaluation and Reflection

The particle system effectively enhanced the readability of the collision by providing a visual cue for the transfer of force. This contributed to the overall clarity of the animation and reinforced the cause-and-effect relationship between the ball and the character. However, the particles do not physically interact with the environment in this implementation, meaning their motion is only an approximation of real physical behaviour. Highlighting a limitation of many real-time particle systems, where visual plausibility is prioritised over full physical accuracy.

The use of particles supported the principle of exaggeration by amplifying the impact event and making it more visually engaging. It also contributed to follow-through by extending the visual effect of the collision beyond the initial contact. With more time, additional features such as collision-enabled particles or more complex simulations (e.g., debris or dust interaction) could be explored to improve realism.

References (Assets)

1. Ringo Gunther, "15th - Fall // Banana Peel," Sketchfab, <https://sketchfab.com/3d-models/15th-fall-banana-peel-60a3ba892429433d9b580998b1036c55>.

References (Documentations/Literature)

2. Epic Games, "Physics in Unreal Engine," Unreal Engine Documentation, <https://dev.epicgames.com/documentation/en-us/unreal-engine/physics-in-unreal-engine#ragdoll-physics>.
3. I. Millington, "*Game Physics Engine Development: How to Build a Robust Commercial-Grade Physics Engine for your Game.*", London: CRC Press, 2010, http://www.r-5.org/files/books/computers/algo-list/realtime-3d/Ian_Millington-Game_Physics_Engine_Development-EN.pdf.
4. Epic Games, "Overview of Niagara Effects for Unreal Engine," Unreal Engine Documentation, <https://dev.epicgames.com/documentation/en-us/unreal-engine/overview-of-niagara-effects-for-unreal-engine>

References (Tutorials)

5. Gorka Games, "How to Animate a Sequence in Unreal Engine 5 (Easy)", *YouTube*, <https://www.youtube.com/watch?v=DYf8i32aa8U>.
6. Unreal University, "How To Make Animations For The Unreal Engine 5 Mannequin", *YouTube*, <https://www.youtube.com/watch?v=tnZv7KQPai0>.
7. Motion Forge CG, "Unreal Engine: Activate Physics in Sequencer & Modify via Sequencer Blueprint", *YouTube*, <https://youtu.be/H7KXnrkpet4>.
8. Aashish, "Quick Tips to Control your Niagara System in Sequencer | Realtime VFX | Unreal Engine 5.4", *YouTube*, <https://youtu.be/9CinSUKV-Vc>.